DESARROLLO DE APLICACIONES II

Guía resumida del TP Integrador

Qué tenemos que hacer, en qué orden y qué debe estar listo en cada etapa

Objetivo del proyecto: construir en grupo un sistema empresarial real, basado en componentes, que crezca entrega a entrega y termine integrando arquitectura en capas, patrones de diseño, seguridad, mensajería asincrónica, SOAP, REST y transacciones.

Documento preparado a partir de los 3 PDFs de la cátedra.

1. Qué hay que lograr al final

El TP no consiste solamente en programar funcionalidades. La cátedra evalúa que el grupo pueda diseñar una arquitectura coherente, justificar las decisiones técnicas y demostrar el sistema funcionando en vivo.

- ☐ Mínimo 6 componentes de negocio identificados y documentados.
- ☐ Al menos 1 componente stateful y 1 stateless, correctamente justificados.
- ☐ Arquitectura en capas: presentación, negocio y datos.
- ☐ Al menos 3 patrones de diseño distintos, aplicados y defendibles.
- ☐ Al menos 1 integración SOAP con un sistema externo legado.
- ☐ Al menos 1 integración REST con un sistema o partner moderno.
- ☐ 2 procesos asincrónicos: 1 con cola punto a punto y 1 con tópico publicación/suscripción.
- ☐ Seguridad declarativa: autenticación y autorización por roles en operaciones sensibles.
- ☐ Transacciones declarativas en al menos un flujo crítico de varios pasos.
- ☐ Repositorio Git con evolución real y participación visible de todos.
- ☐ Tablero Jira (o equivalente aprobado) actualizado durante todo el proyecto.
- ☐ Documentación técnica, documentación funcional, manual de usuario y prototipo navegable.
- ☐ Aplicación responsive si el sistema es web.
- ☐ Todos los integrantes deben poder explicar cualquier parte del sistema.

2. Orden recomendado de trabajo

Paso 1 - Armar el equipo y elegir el dominio

- Trabajar en un grupo estable de 3 a 4 integrantes.
- Elegir un único dominio para todo el cuatrimestre: MediConecta, LogiRed, StayHub o un dominio propio aprobado.
- MediConecta es el caso recomendado por la cátedra porque tiene mayor nivel de detalle y menor ambigüedad.

Paso 2 - Crear la base de organización

- Crear el repositorio Git desde el principio.
- Crear y mantener un tablero Jira, Trello o equivalente validado.
- Registrar tareas, responsables y avances reales; no actualizarlo solamente antes de las entregas.
- Dar acceso de lectura al docente cuando corresponda.

Paso 3 - Diseñar la arquitectura antes de programar todo

- Identificar al menos 6 componentes de negocio.
- Definir para cada componente: nombre, responsabilidad, operaciones e interfaz.
- Hacer un diagrama general con componentes y dependencias.
- Elegir y justificar el stack tecnológico.

- Decidir qué componente será stateful y cuáles stateless.

Paso 4 - Preparar el entorno y levantar el primer componente

- Si usan Jakarta EE: JDK 17+, Eclipse Enterprise, Maven y WildFly.
- Levantar WildFly y comprobar que funcione en localhost.
- Crear un proyecto Maven empaquetado como WAR.
- Desplegar el componente en WildFly y comprobarlo desde el navegador.
- La guía de la cátedra usa como prueba el mensaje: '¡Hola desde el contenedor!'.

Paso 5 - Convertir la prueba técnica en el primer componente real

- Implementar un componente real del dominio.
- Separarlo claramente en presentación, negocio y datos.
- Desplegarlo en un contenedor real.
- Justificar si es stateful o stateless.
- Preparar la documentación inicial de arquitectura.

Paso 6 - Ampliar a 3 componentes y aplicar diseño

- Llegar a al menos 3 componentes implementados y desplegados.
- Tener por lo menos un stateful y un stateless.
- Aplicar y justificar al menos 3 patrones de diseño.
- Agregar seguridad declarativa mínima.

Paso 7 - Diseñar e implementar integración asincrónica

- Definir qué procesos conviene resolver de manera asincrónica.
- Configurar un broker como ActiveMQ, RabbitMQ o equivalente.
- Implementar primero un flujo real con cola o tópico.
- Luego completar los dos mecanismos requeridos: cola punto a punto y tópico pub/sub.

Paso 8 - Agregar integraciones síncronas

- Implementar al menos un servicio SOAP y su WSDL para representar un sistema legado.
- Implementar al menos un servicio REST para un partner moderno o API externa.
- Justificar por qué cada integración es síncrona o asincrónica.

Paso 9 - Incorporar transacciones y seguridad completa

- Agregar transacciones declarativas a un flujo crítico de varios pasos.
- Asegurar al menos 2 operaciones sensibles con autenticación y autorización por rol.
- Poder explicar qué ocurre si el flujo falla a mitad de camino.

Paso 10 - Completar documentación, diseño y experiencia de usuario

- Documentación funcional: casos de uso o historias de usuario.
- Manual de usuario final.
- Prototipo navegable de pantallas principales en Figma o InVision.
- Diseño responsive en web.
- Temas claro, oscuro y estándar son opcionales y suman como extra si funcionan en tiempo real.

Paso 11 - Integrar todo y preparar la defensa

- Probar un flujo de negocio de punta a punta.
- Actualizar diagramas y documentación según lo realmente implementado.
- Revisar Git y Jira para que reflejen la evolución del trabajo.
- Practicar preguntas cruzadas: cada integrante debe poder explicar cualquier parte del proyecto.

3. Entregas: qué debe crecer en cada etapa

La regla más importante es que cada entrega debe agregar valor real respecto de la anterior. No alcanza con repetir la misma demo con más documentación.

Etapa	Objetivo principal	Qué debería verse funcionando
Entrega 1	Primer componente real	Componente desplegado, arquitectura en capas y funcionamiento en vivo.
Entrega 2	Varios componentes + diseño	Al menos 3 componentes, patrones de diseño y seguridad declarativa.
Entrega 3	Mensajería asincrónica	Primer proceso de negocio real usando cola o tópico.
Entrega 4	Integración completa	SOAP + REST + mensajería completa + transacciones.
Entrega final	Sistema completo	Flujo end-to-end, documentación consolidada y defensa oral grupal e individual.

4. Patrones de diseño

No alcanza con nombrar tres patrones. Cada integrante tiene que poder explicar qué problema resuelve cada uno en el dominio elegido y por qué se descartaron otras alternativas.

- **DAO:** Separar acceso a datos de la lógica de negocio.
- **Facade:** Simplificar el acceso a un componente o subsistema complejo.
- **Adapter:** Unificar integraciones heterogéneas, por ejemplo SOAP y REST.
- **Factory:** Centralizar la creación de objetos que varían según proveedor, canal o tipo.
- **Strategy:** Cambiar algoritmos o políticas en tiempo de ejecución.

5. Guía rápida de Jakarta EE / WildFly

☐ Instalar JDK 17 o superior.

☐ Usar Eclipse IDE for Enterprise Java and Web Developers.

☐ Descomprimir WildFly en una ruta simple, evitando OneDrive, espacios y tildes si es posible.

☐ Ejecutar standalone.bat y verificar http://localhost:8080.

☐ Crear usuario de administración con add-user.bat y probar http://localhost:9990.

☐ Crear proyecto Maven simple.

☐ Configurar packaging WAR y dependencia jakarta.jakartaee-api.

☐ Crear beans.xml.

☐ Crear las clases del componente.

☐ Ejecutar Maven: clean package.

☐ Desplegar el WAR desde la consola de WildFly.

☐ Probar la URL final del recurso desde el navegador.

6. Documentación y materiales que no conviene dejar para el final

☐ Diagrama de arquitectura general.

☐ Interfaces y responsabilidades de los componentes.

☐ Documento técnico actualizado en cada etapa.

☐ Diagramas de secuencia de las integraciones.

☐ Casos de uso o historias de usuario.

☐ Manual de usuario.

☐ Prototipo navegable en Figma o InVision.

☐ Repositorio Git con commits incrementales.

☐ Tablero Jira actualizado.

☐ Declaración del uso de IA generativa: qué se usó y para qué.

7. Qué se evalúa

Criterio	Peso
Diseño arquitectónico y coherencia de componentes	**25%**
Correctitud funcional e implementación	**25%**
Patrones e integración correctamente justificados	**20%**
Calidad de la documentación	**15%**
Defensa oral y comprensión individual	**15%**

8. Punto crítico: las fechas se contradicen

IMPORTANTE: los PDFs no muestran exactamente las mismas fechas. El TP general indica un cronograma y el anexo muestra otro, aunque el propio anexo dice que no modifica las fechas previas. Por lo tanto, el grupo debería confirmar con el docente o por Teams cuál cronograma está vigente antes de organizar las entregas.

Instancia	TP general	Anexo
Entrega / checkpoint 1	31/08	04/09
1.er parcial obligatorio	14/09	18/09
Checkpoint 2	19/10	23/10
2.º parcial obligatorio	09/11	13/11

9. Checklist final para el grupo

☐ Tenemos un dominio definido y aprobado.

☐ Tenemos mínimo 6 componentes claramente separados.

☐ Tenemos diagrama de arquitectura.

☐ Tenemos al menos 1 stateful y 1 stateless.

☐ Cada componente implementado respeta presentación / negocio / datos.

☐ Aplicamos y justificamos 3 patrones de diseño.

☐ Tenemos SOAP + WSDL.

☐ Tenemos REST.

☐ Tenemos una cola punto a punto.

☐ Tenemos un tópico publicación/suscripción.

☐ Tenemos seguridad por roles.

☐ Tenemos transacciones en un flujo crítico.

☐ Git muestra trabajo incremental de todos.

☐ Jira está actualizado.

☐ Tenemos documentación funcional, manual y prototipo.

☐ La interfaz es responsive si es web.

☐ Todos podemos explicar cualquier parte del sistema.

☐ La demo funciona realmente de punta a punta.

10. Resumen ultra corto para repartir tareas

Ahora	Después
Grupo + dominio + Git + Jira + arquitectura + 6 componentes + primer componente desplegado.	3 componentes + patrones + seguridad -> mensajería -> SOAP/REST -> transacciones -> documentación -> integración final.

Consejo de organización: no dividir el proyecto en 'cada uno sabe solo su parte'. Conviene rotar revisiones y hacer demos internas porque la defensa es individual sobre cualquier parte del sistema.